

A Graph Propagation-Based Automatic Differentiation Method for Power Electronic and Electric Machine Systems Simulation

Shengyu Jia, *Student Member, IEEE*, Bochen Shi, *Member, IEEE*, Zhengming Zhao, *Fellow, IEEE*, Liqiang Yuan, *Member, IEEE*

Abstract—This paper proposes an automatic differential method to enable the effective Taylor-series based flexible integration algorithm for power electronics and electric machine systems simulation. The proposed approach decomposes the system equations into linear and non-linear parts and encapsulates the non-linear part through controlled-source interfaces. Furthermore, this paper introduces a graph propagation-based automatic differentiation method (GP-AD), which enables the automatic generation of higher-order derivatives for nonlinear portion. To evaluate the efficiency of the proposed approach, a case study on a power traction system of a high-speed train is presented. The results indicate that the proposed algorithm achieves a 6-fold speedup compared to the commercial software, demonstrating the effectiveness of the proposed method.

Index Terms—Higher-order derivatives, automated differentiation, adaptive Taylor series based integration, nonlinear, power electronics.

I. INTRODUCTION

Modern electric machine (EM) drive systems with power electronics (PE) devices often exhibit a mixed structure that combines linear and non-linear components [1], which poses challenges for existing simulation integration algorithms.

Various algorithms can solve nonlinear ODEs, but they often struggle with inefficiencies in power electronics simulations due to limited flexibility [2]. Among methods leveraging linear properties for acceleration [3]–[5], the Taylor series-based flexible adaptive (FA) algorithm [4] dynamically adjusts integration step size and order over varying time intervals, making it particularly effective for power electronics systems simulation.

However, extending the FA algorithm to PE-EM systems poses significant challenges in obtaining higher-order derivatives for nonlinear equations [6]. Manual symbolic differentiation of nonlinear machine equations is complex and error-prone [7]. While automatic differentiation (AD) techniques have been proposed, existing implementations have limitations. Numerical differentiation methods [8], [9] suffer from numerical errors and restricted derivative orders due to error accumulation. Symbolic expression-based AD tools [10] offer high precision but incur high computational costs, making them inefficient for PE-EM simulations.

Thus, this paper aims to propose an efficient and accurate automatic higher-order derivative computation method,

This work was supported by National Key R&D Program of China (2023YFB3307000) and National Natural Science Foundation of China (52307211).

The authors are with the Department of Electrical Engineering, Tsinghua University, Beijing 100084, China (E-mail: jsy20@mails.tsinghua.edu.cn; sbc@mail.tsinghua.edu.cn; zhaozm@tsinghua.edu.cn; yllq@tsinghua.edu.cn).

Manuscript received XX; revised XX.

enabling the FA algorithm's automatic implementation in PE-EM systems. The approach decomposes system equations into linear and nonlinear components, encapsulating nonlinearity through controlled sources. The proposed graph propagation-based automatic differentiation (GP-AD) method represents nonlinear equations as a directed graph, propagating derivatives along graph paths via the chain rule. By integrating GP-AD, the FA algorithm adaptively computes higher-order derivatives for state and algebraic variables while adjusting step size and integration order through error control, ensuring high-efficiency simulation for PE-EM systems.

II. SYSTEM DECOUPLED MODELING FOR LINEAR AND NON-LINEAR SUBSYSTEMS WITH ALGEBRAIC INTERFACES

Generally, the system can be represented with the state space equations:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}), \quad (1a)$$

$$\mathbf{y} = \mathbf{g}(\mathbf{x}, \mathbf{u}) \quad (1b)$$

where $\mathbf{x} \in \mathbb{R}^{n \times 1}$ denotes the state variables, $\mathbf{u} \in \mathbb{R}^{m \times 1}$ includes inputs such as voltage and current source. $\mathbf{y} \in \mathbb{R}^{l \times 1}$ denotes the output vector. $\mathbf{f} : \mathbb{R}^{n \times 1} \times \mathbb{R}^{m \times 1} \rightarrow \mathbb{R}^{n \times 1}$ represents the state equations, and $\mathbf{g} : \mathbb{R}^{n \times 1} \times \mathbb{R}^{m \times 1} \rightarrow \mathbb{R}^{l \times 1}$ denotes the output equations.

For linear systems, the function $\mathbf{f}(\mathbf{x}, \mathbf{u})$ can typically be expressed in the form $\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, where $\mathbf{A}$ and $\mathbf{B}$ are system coefficient matrices. However, for nonlinear components such as electric machines, the presence of factors like electromagnetic torque and magnetic saturation introduces non-additive and nonlinear behaviors in the state equations. As a result, these equations cannot be represented in linear form, which accounts for the increased complexity in computing higher-order derivatives.

Although machine equations appear to be nonlinear overall, it can be separated to the linear and nonlinear parts by introducing controlled sources in the equivalent circuit modeling. For instance, as shown in Fig. 1, the dq-axis equivalent circuit of an induction machine primarily consists of RLC components, which can be treated as linear. The nonlinear effects, including electromagnetic torque, back-EMF, and dq transformation, are incorporated into algebraic equations through controlled sources and measurement outputs as controlling quantities.

Thus, the controlled source acts as an “algebraic interface” linking the linear and nonlinear parts. Each interface is defined by an output $\mathbf{u}_c$ driven by measured signals $\mathbf{y}_m$:

$$\mathbf{u}_c = \mathbf{h}(\mathbf{y}_m), \quad (2)$$

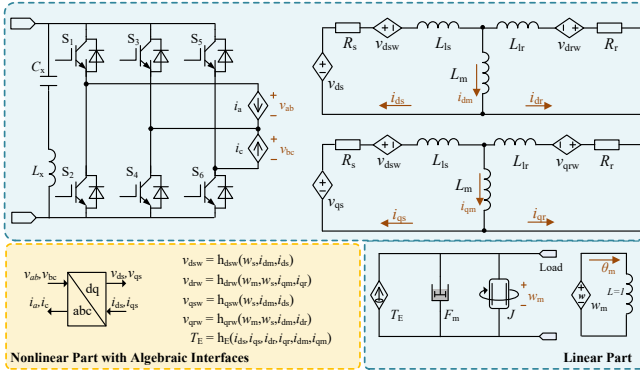


Fig. 1. Schematic of a motor drive system, illustrating the decomposition into a linear subsystem and algebraic nonlinear interfaces.

where $h(\cdot)$ is constructed from elementary operations (e.g., addition, multiplication, trigonometric functions).

This modeling technique effectively splits the machine's nonlinear equations by placing all differential equations in the linear part and only algebraic equations in the nonlinear portion. Then, the model can be expressed as Eq.(3) and (4).

$$\dot{x} = A_k x + \begin{bmatrix} B_k^1 & B_k^2 \end{bmatrix} \begin{bmatrix} u_e \\ u_c \end{bmatrix}, \quad (3a)$$

$$y = C_k x + \begin{bmatrix} D_k^1 & D_k^2 \end{bmatrix} \begin{bmatrix} u_e \\ u_c \end{bmatrix}, \quad (3b)$$

with u_c determined by

$$u_c = h_{nl}(y_m), \quad (4a)$$

$$y_m = C_{mk} x + D_{mk} u_e. \quad (4b)$$

where Eq.(3) denotes the linear part, and Eq.(4) denotes the nonlinear part. $u_c \in R^{m_c \times 1}$ denotes the controlled source (as the role of algebraic interface) and $u_e \in R^{m_e \times 1}$ denotes the independent input. $y_m \in R^{l_m \times 1}$ is the measurement output associated with u_c . A_k, B_k, C_k, D_k are matrices determined by circuit topology and parameters, and k denotes different topologies decided by switch states. $h_{nl} : R^{l_m \times 1} \rightarrow R^{m_c \times 1}$ represents the nonlinear equations in the system. C_{mk}, D_{mk} are matrices related to calculation of y_m .

This modeling technique separates the PE-EM system's linear and nonlinear equations using an algebraic interface, enabling independent treatment of the two parts. This provides the foundation for the subsequent algorithm implementation.

III. GRAPH PROPAGATION-BASED AUTOMATIC DIFFERENTIATION AND FLEXIBLE ADAPTIVE INTEGRATION ALGORITHM

Before introducing the specific method, the overall solution procedure for the system equations is first briefly outlined. By decoupling the system equations as expressed in Eq. (3) and (4), a semi-explicit differential algebraic equation (DAE) [12] formulation is obtained. This allows a stepwise solution strategy at each time step t_n :

- 1) Solve Eq. (3) to obtain x at t_{n+1} with order $= p$; and update the needed measurement output y_m of p th order with Eq. (4b).

- 2) Solve the nonlinear part Eq. (4a) (i.e., algebraic interface) to update p th order u_c at t_{n+1} .

Building on this foundation, this section then presents the GP-AD method and its integration into the FA algorithm.

A. Nonlinear Modeling with Elementary Operations to form the Directed Graph

To compute higher-order derivatives of the algebraic interface u_c , we first derive the measurement vector y_m 's derivatives by differentiating (4b):

$$y_m^{(i)}(t) = C_{mk} x^{(i)}(t) + D_{mk}^1 u_e^{(i)}(t). \quad (5)$$

Since Eq. (4a) defines u_c as a function of y_m , once $y_m^{(i)}$ is obtained, it can be substituted into h to compute the corresponding derivative of u_c .

Since the function $h_{nl}(\cdot)$ is composed of elementary operations, it can be represented as a computation graph consisting of these operations. The derivatives can then be computed locally for each operation and subsequently combined to obtain the full derivative. By structuring h in this way, an element-wise directed graph (DG) is constructed, enabling derivative propagation from y_m to u_c . This modeling approach is illustrated in Fig. 2, where the red elements denote individual elementary operations.

The directed graph is denoted as $G(V, E)$. The nodes V of this graph consist of the input variables y_m , the output variables u_c , and intermediate computational nodes $h_i(\cdot)$, while the edges E represent the data flow between these nodes. The depth-first search (DFS) algorithm [13] is used to determine the computational dependencies by identifying paths from output nodes u_c back to input nodes y_m , generating computational chains $\mathcal{P} = P_i$, where each path P_i represents a sequence of operations.

B. Automatic Derivative via Graph Propagation

Once the computational paths $\mathcal{P}$ are identified, higher-order derivatives of u_c can be propagated along these paths. Let v_j represent the output of a node along a path $P_i \in \mathcal{P}$, and let h_j represent the operation associated with v_j . For single-input functions (e.g., cos, exp), the q th-order derivative follows the Faà di Bruno formula [14] which is a generalizing of chain rule to higher derivatives:

$$\frac{d^q v_j}{dt^q} = \sum_{r=1}^q h^{(r)}(v_{j-1}) B_{q,r} \left(\frac{dv_{j-1}}{dt}, \dots, \frac{d^{q-r+1} v_{j-1}}{dt^{q-r+1}} \right), \quad (6)$$

where $B_{q,r}$ are the partial exponential Bell polynomials, which determine the composition of the higher-order derivatives of v_{j-1} .

For multi-input functions (e.g., multiplication, division), following the chain rule, the q th-order derivative is:

$$\frac{d^q v_j}{dt^q} = \sum_{r=0}^q C_{q,r} \prod_{k=1}^{m_j} \frac{d^{q_{r,k}} v_{j,in,k}}{dt^{q_{r,k}}} \quad (7)$$

where $v_{j,in,k}$ denotes the k -th input node to v_j , and m_j is the number of inputs to v_j . The term $C_{q,r}$ represents the

combinatorial coefficient for the r -th term in the computation of the q -th derivative (for the case of two inputs, it corresponds to the binomial coefficient). The quantity $q_{r,k}$ denotes the order of differentiation for the k -th input in the r -th term, satisfying $\sum_{k=1}^{m_j} q_{r,k} = q$.

For a specific computational path $P_i = \{y_m = v_0, v_1, v_2, \dots, v_n, u_c = v_{n+1}\}$, the complete derivative of u_c is obtained by appropriately aggregating the derivatives along all paths $P_i \in \mathcal{P}$. The elementary operators are precomputed in a library along with their high-order partial derivatives. Additionally, the previously computed derivatives are stored and reused to reduce computational redundancy. Overall, the GP-AD method is shown in Fig. 2.

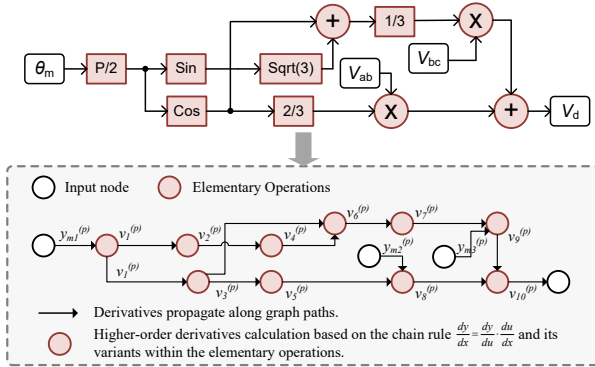


Fig. 2. Graph Propagation-Based Automatic Differentiation Diagram

C. the FA Integration Algorithm and whole Flowchart

Suppose that for the state variable x , all derivatives at t_{n-1} are known. Based on the Taylor series expansion, the numerical solution of the state equation at time t_n is:

$$x_n = x_{n-1} + \sum_{i=1}^p \frac{x_{n-1}^{(i)}}{i!} (\Delta t_n)^i, \quad (8)$$

where Δt_n is the step size for the n -th step, $x_{n-1}^{(i)}$ is the i -th time derivative of x at t_{n-1} , and p is the order at which the series is truncated.

The local truncation error (LTE) at t_n is the sum of higher-order terms beyond p and can be approximated by its leading term (the $(p+1)$ -th order):

$$\varepsilon_n \approx \frac{x_{n-1}^{(p+1)}}{(p+1)!} (\Delta t_n)^{p+1}. \quad (9)$$

Thus, the FA algorithm has an LTE of $O(p+1)$.

Bringing the FA integration algorithm and the GP-AD technique together, the overall flowchart of solving the PE-EM systems is shown in Fig.3.

D. Algorithm Error and Numerical Stability Analysis

We first analyze the LTE, which consists of two components: the integration error from the FA algorithm and the

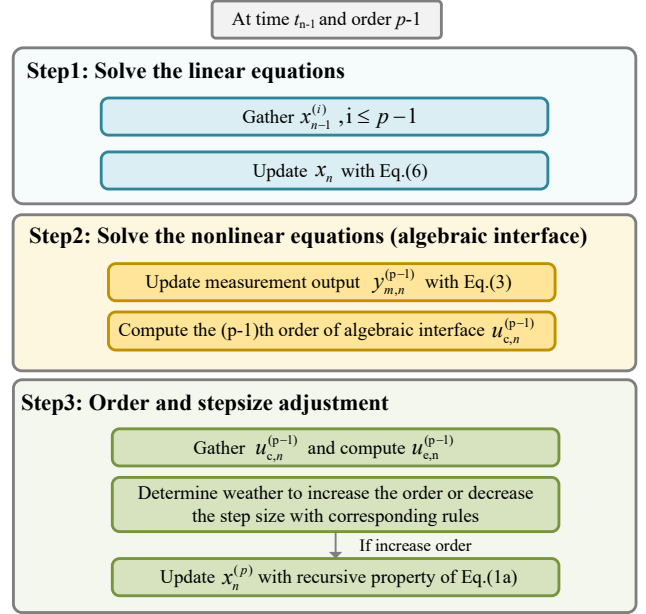


Fig. 3. Flowchart of the FA Integration Algorithm with the proposed GP-AD Approach.

differentiation error from the GP-AD process. The LTE of the p -th order FA method is given by Eq.(9). Defining the numerical error as $x_{\text{num}} - x_{\text{exact}} = \varepsilon_x = O(\Delta t_n^{p+1})$, where the subscripts num and exact denote the numerical and exact solutions, respectively. Then, considering the proposed PG-AD approach, the LTE can be derived as follows.

From the measurement calculation Eq.(4b), the measurement error is:

$$\Delta y_m = y_{m,\text{num}} - y_{m,\text{exact}} = C_{mk} \varepsilon_x. \quad (10)$$

Since h is a smooth function, its Taylor series expansion around $y_{m,\text{exact}}$ gives:

$$h(y_{m,\text{exact}} + \Delta y_m) = h(y_{m,\text{exact}}) + h'(y_{m,\text{exact}}) \Delta y_m + O(\|\Delta y_m\|^2). \quad (11)$$

The algebraic interface error is defined as

$$\varepsilon_{u_c} = u_{c,\text{num}} - u_{c,\text{exact}} = h(y_{m,\text{num}}) - h(y_{m,\text{exact}}). \quad (12)$$

Substituting the Taylor expansion, we obtain

$$\varepsilon_{u_c} \approx h'(y_{m,\text{exact}}) \Delta y_m = h'(y_{m,\text{exact}}) C_{mk} \varepsilon_x. \quad (13)$$

Since $\varepsilon_x = O(\Delta t_n^{p+1})$, it follows that

$$\varepsilon_{u_c} = O(\Delta t_n^{p+1}). \quad (14)$$

Thus, both the linear and nonlinear interface maintain an error order of $p+1$:

$$\varepsilon_{\text{total}} = \|x_{\text{num}} - x_{\text{exact}}\| + \|u_{c,\text{num}} - u_{c,\text{exact}}\| = O(h^{p+1}). \quad (15)$$

Secondly, regarding numerical stability, the proposed GP-AD method does not introduce additional errors and therefore does not negatively affect the overall numerical stability. The stability characteristics and stability region of the algorithm are primarily determined by the FA method, which shares the same stability properties as typical forward integration schemes [4].

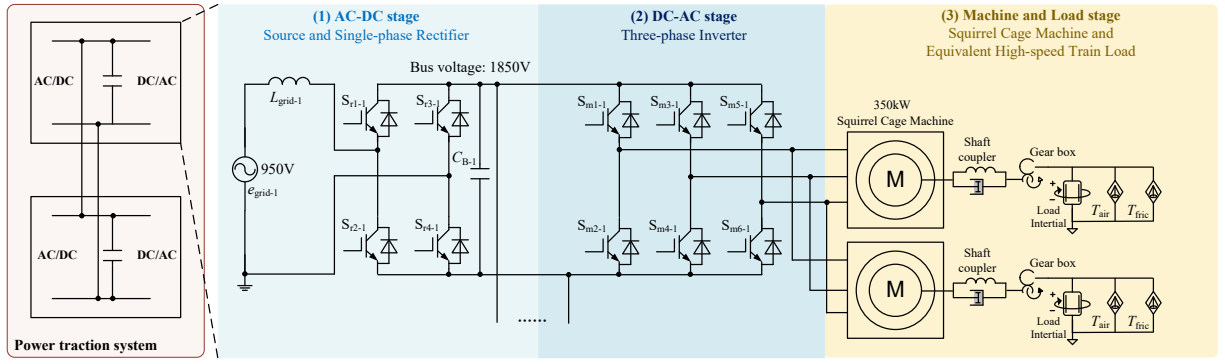


Fig. 4. Topology diagram of the traction converter driving the induction motor.

IV. CASE STUDY

To evaluate the simulation accuracy and efficiency of the proposed method, this section presents a case study based on the power traction system of a high-speed train. High-speed train traction is a representative and important industrial application of PE-EM systems. Typically, a high-speed train consists of multiple carriages, with each traction converter driving several induction motors—forming a typical PE-EM system that includes both linear and nonlinear components. This system configuration directly aligns with the target application scenarios addressed by the proposed method. Therefore, by analyzing the simulation performance of this case, the effectiveness and practicality of the proposed method can be validated.

The topology of the traction converter are shown in Fig. 4. The simulation captures a scenario where the train accelerates from rest to a medium speed (approximately 250 km/h), with a constant acceleration of 0.5 m/s^2 . The motor shaft is connected to the load through a coupling and a gearbox. The load consists of three components: inertia load, mechanical resistance load, and aerodynamic resistance load. The key system parameters are listed in Table I.

TABLE I
PARAMETERS OF THE HIGH-SPEED TRAIN TRACTION SYSTEM

Parameter	Value
Input AC voltage (RMS) U_a	950V
DC bus voltage U_c	1850V
Rated power per motor P_e	350kW
Rated torque per motor T_e	800Nm
Air resistance torque coefficient K_a	0.254
Mechanical resistance torque T_f	66Nm
Load Rotational inertia	2600kg·m ²

Fig. 5 presents overall simulation results, including the rotor speed, mechanical load torque, electromagnetic torque, and DC-link voltage. To verify the accuracy, the proposed method is compared against results obtained from a commercial software. Furthermore, Fig.6 compares the transient waveforms of different parts of the system at various dynamic moments. These include the stator flux and current during the start-up

process or in a short period after steady state is reached, and the dynamic response of mechanical load torque and electromagnetic torque following the end of the acceleration phase.

Then, to quantitatively compare the results, absolute and relative error evaluations for several key quantities are presented. The detailed error comparison results are summarized in Tables II and III. In Table II, err_{rel} denotes the relative error, and err_{abs} denotes the absolute error. For metrics such as current THD and normalized ripple RMS, which have relatively small magnitudes, the comparisons are performed using absolute error. Other quantities are compared using either relative error. In Table III, the relative error for waveform time series was computed point-by-point over time, using the commercial software results as the reference.

Based on the above error analysis and waveform comparisons, it can be observed that the proposed method achieves a high degree of consistency with the commercial software for most waveform results. However, some discrepancies—with relative errors around 3%—can still be seen in certain cases, along with minor differences in transient waveforms. The main reason for these differences is that due to inherent differences between the two machine models, exact equivalence is difficult to achieve. Overall, when these results are viewed in the context of the full analysis and comparison, the differences remain within an acceptable range, and the outcomes confirm the accuracy and validity of the proposed method.

Table IV provides a comparison of simulation efficiency among the proposed automated derivative-FA algorithm within the DSED framework, the Dormand–Prince (RK45) algorithm within the DSED framework, and the RK45 algorithm implemented in commercial software. In this letter, PLECS is employed as the commercial benchmark for simulation comparison. Within PLECS, the DOPRI solver, which is essentially a RK45 algorithm, is used for numerical integration. The relative error setting of the solvers are all $1e-6$. All three methods simulate a 150s dynamic process.

We briefly note that in the DSED framework, a semi-symbolic modeling approach [15] is adopted to represent switching devices, allowing the coefficient matrices to be updated dynamically as the switching states change. As this work primarily focuses on the overall system-level behavior,

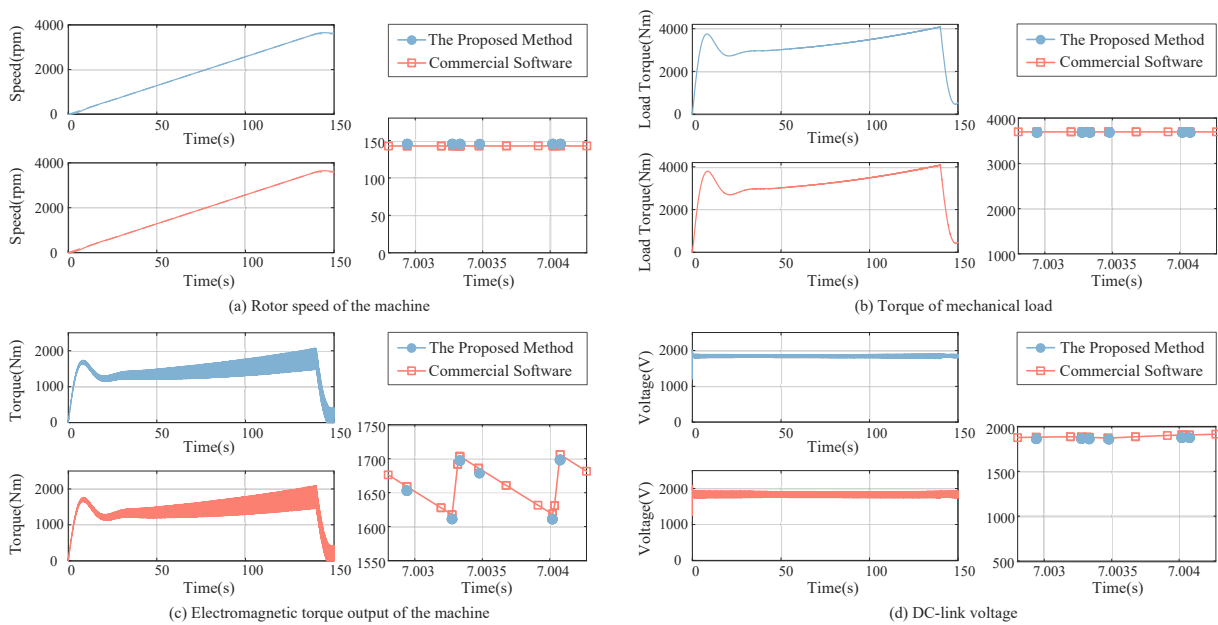


Fig. 5. Comparisons between the simulated results.

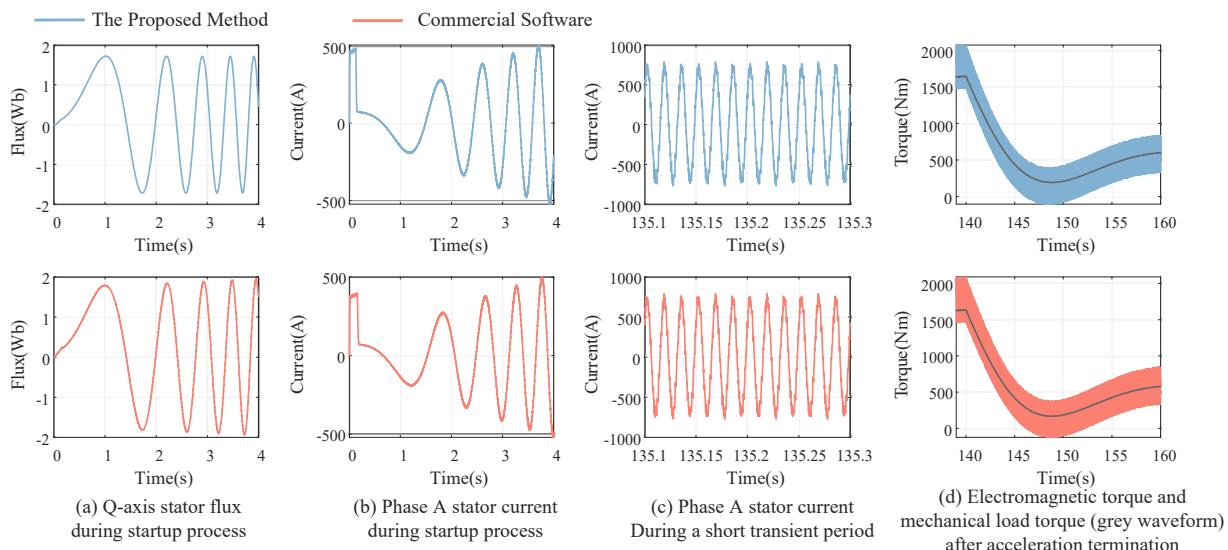


Fig. 6. Transient waveforms of different parts and scenarios.

switching transients are not explicitly modeled. However, if detailed transient modeling is required, the Piecewise Analytical Transient (PAT) model [16], [17] can be employed to efficiently simulate the switching process.

It can be observed from Table IV that the proposed method achieves a 6-fold speedup compared to the commercial software. The table also provides a breakdown of the simulation time, including the total integration time and the time spent computing the nonlinear algebraic interface within the proposed method. The results indicate that the computational cost for nonlinear part accounts for only about 1/3 of the total simulation time. This further demonstrates the effectiveness of the proposed approach.

V. CONCLUSION

This paper proposes a graph propagation-based automatic differentiation (GP-AD) method integrated with a flexible adaptive (FA) integration algorithm for simulating power electronics and electric machine systems. GP-AD automates higher-order derivative computation using graph propagation and chain rules, with precomputed expressions enhancing efficiency. Combined with the variable-step, variable-order FA algorithm and separately handling the linear and nonlinear part, it ensures accurate and efficient numerical integration. A case study on a high-speed train traction system shows a sixfold speedup over commercial simulation software while maintaining accuracy, demonstrating the method's effectiveness.

TABLE II
COMPARISON OF CURRENT AND CONTROL PERFORMANCE

Method	Speed control peak error time	Speed control settling time	Load stabilization time (from 140s)	THD of A phase stator current	Normalized Ripple RMS
The proposed method	15.61s	23.85s	26.81s	0.09%	2.80%
Commercial software	15.79s	24.10s	27.77s	0.13%	2.44%
Absolute/Relative error	$err_{rel}=1.14\%$	$err_{rel}=1.07\%$	$err_{rel}=3.46\%$	$err_{abs}=0.04\%$	$err_{abs}=0.36\%$

TABLE III
OVERALL RELATIVE ERROR

Variable	Relative error
Electromagnetic torque	3.42%
Mechanical load	0.94%
Rotational speed	0.74%
Time of switching actions	0.15%

TABLE IV
PERFORMANCE COMPARISON OF DIFFERENT ALGORITHMS

Metric	DSED/FA	DSED/RK45	Commercial Soft- ware/RK45
Total CPU time (s)	59.24	187.20	354.26
Normalized CPU time	1	3.16	5.98
Number of computation points	9.02×10^5	1.06×10^6	2.58×10^6
CPU time per step (us)	65.68	176.60	137.31
Total time for solving nonlinear part (s)	18.97	56.15	\

In the following, we discuss the computational efficiency and practical application scenarios of the proposed method. The GP-AD method, combined with the FA integration algorithm, enhances simulation efficiency by adaptively adjusting the integration order in systems with frequent discrete events. Since the FA algorithm is essentially a forward integration method, the proposed approach demonstrates significant acceleration benefits in the simulation of non-stiff systems. These benefits are particularly prominent in large-scale circuits with frequent switching events, where the ability to flexibly vary the integration order enables better adaptation to the system's dynamic behavior and variable step-size constraints. From an industrial perspective, the proposed method is particularly applicable to scenarios involving power electronic and electric machine systems, as well as components such as photovoltaics and batteries. It is also well-suited for modeling systems where magnetic saturation effects must be considered, such as in electric machines and transformers.

REFERENCES

[1] A. Medina, A. Ramos-Paz, and C. Fuente-Esquivel, "Periodic steady state solution of electric systems with nonlinear components using parallel processing," *IEEE Transactions on Power Systems*, vol. 18, no. 2, pp. 963–965, May 2003.

[2] L. Petzold, "Automatic Selection of Methods for Solving Stiff and Nonstiff Systems of Ordinary Differential Equations," *SIAM Journal on Scientific and Statistical Computing*, Jul. 2006.

[3] C.-C. Liu, J. Hsieh, C. Chang, J. Bocek, and Y.-T. Hsiao, "A fast-decoupled method for time-domain simulation of power converters," *IEEE Transactions on Power Electronics*, vol. 8, no. 1, pp. 37–45, Jan. 1993.

[4] Y. Zhu, Z. Zhao, B. Shi, and Z. Yu, "Discrete State Event-Driven Framework With a Flexible Adaptive Algorithm for Simulation of Power Electronic Systems," *IEEE Transactions on Power Electronics*, vol. 34, no. 12, pp. 11 692–11 705, Dec. 2019.

[5] V. Satek, P. Veigend, and G. Necasova, "Taylor Series Based Integration in Electric Circuits Simulations," *Advances in Electrical and Electronic Engineering*, vol. 17, no. 3, pp. 352–359, Sep. 2019.

[6] A. Ghorbani and M. Gachpazan, "A high-order algorithm for solving nonlinear algebraic equations," *Iranian Journal of Numerical Analysis and Optimization*, vol. 11, no. 1, pp. 107–115, Mar. 2021.

[7] Y. Ma, V. Dixit, M. J. Innes, X. Guo, and C. Rackauckas, "A Comparison of Automatic Differentiation and Continuous Sensitivity Analysis for Derivatives of Differential Equation Solutions," in *2021 IEEE High Performance Extreme Computing Conference (HPEC)*. Waltham, MA, USA: IEEE, Sep. 2021, pp. 1–9.

[8] H. Xu, B. Shi, Z. Yu, J. Zheng, and Z. Zhao, "Numerical Derivative-Based Flexible Integration Algorithm for Power Electronic Systems Simulation Considering Nonlinear Components," *IEEE Transactions on Industrial Electronics*, pp. 1–11, 2023.

[9] E. Miletics and G. Molnárka, "Taylor series method with numerical derivatives for initial value problems," *Journal of Computational Methods in Sciences and Engineering*, vol. 4, no. 1–2, pp. 105–114, Jan. 2004.

[10] U. Schmitt, B. Moser, C. S. Lorenz, and A. Réfrégier, "SymPy2c: From symbolic expressions to fast C/C++ functions and ODE solvers in Python," *Astronomy and Computing*, vol. 42, p. 100666, Jan. 2023.

[11] E. Hairer and G. Wanner, *Solving Ordinary Differential Equations II*, ser. Springer Series in Computational Mathematics. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, vol. 14.

[12] M. Arnold, K. Strehmel, and R. Weiner, "Half-explicit Runge-Kutta methods for semi-explicit differential-algebraic equations of index 1," *Numerische Mathematik*, vol. 64, no. 1, pp. 409–431, Dec. 1993.

[13] S. Even, *Graph Algorithms*. Cambridge University Press, Sep. 2011.

[14] W. P. Johnson, "The Curious History of Faà di Bruno's Formula," *The American Mathematical Monthly*, vol. 109, no. 3, pp. 217–234, Mar. 2002.

[15] Z. Yu, Z. Zhao, B. Shi, Y. Zhu, and J. Ju, "An Automated Semi-symbolic State Equation Generation Method for Simulation of Power Electronic Systems," *IEEE Transactions on Power Electronics*, vol. 36, no. 4, pp. 3946–3956, Apr. 2021.

[16] B. Shi, Z. Zhao, and Y. Zhu, "Piecewise Analytical Transient Model for Power Switching Device Commutation Unit," *IEEE Transactions on Power Electronics*, vol. 34, no. 6, pp. 5720–5736, Jun. 2019.

[17] Y. Xiao, Z. Zhao, B. Shi, Z. Yu, S. Jia, and S. Ji, "Piecewise Analytical Transient Model of SiC MOSFET and SiC Schottky Diode Pair," in *2023 IEEE 24th Workshop on Control and Modeling for Power Electronics (COMPEL)*. Ann Arbor, MI, USA: IEEE, Jun. 2023, pp. 1–6.